

Fix: Uncaught TypeError: Cannot read properties of undefined (reading 'createContext')

Error Description

Symptoms

```
[Inline Polyfill] 🔄 Starting synchronous global patching...
[Inline Polyfill] ✅ window.global = window
[Inline Polyfill] ✅ Patched 3 properties across 1 global objects
[Inline Polyfill] ✅ Destructuring test: PASSED
[Inline Polyfill] 🛑 Complete - Module scripts can now load safely
misc-e9aabc3a-mg7zhgal.js:32 Uncaught TypeError: Cannot read properties of undefined (reading
'createContext')
    at misc-e9aabc3a-mg7zhgal.js:32:255
```

What Happens

- Polyfills load and patch successfully
- Application crashes when `misc` chunk tries to execute
- Error occurs at module parse time (not runtime)
- `React` is undefined when `createContext` is called

Root Cause

The Problem

Chunk loading race condition: Vite splits code into multiple chunks that load in parallel. When the `misc` chunk (containing `i18n` libraries like `react-i18next`) loads and executes **before** the chunk containing `React`, any code calling `React.createContext` fails because `React` is undefined.

Why It Happens

1. **Parallel chunk loading:** Browser loads multiple JS chunks simultaneously
2. **No guaranteed load order:** Vite doesn't enforce which chunk loads first
3. **Module parse-time execution:** `react-i18next` calls `createContext` when the module is parsed, not when it's used
4. **Race condition:** `misc` chunk can parse before `React` chunk loads

Technical Details

```
// In react-i18next source code (runs at module parse time):
import { createContext } from 'react'; // React might not be loaded yet
const I18nContext = createContext(null); // ❌ Error if React undefined
```

When bundled into `misc` chunk and loaded before `React` is available, this crashes.

Solution

Overview

The fix requires **three coordinated changes**:

1. **Lazy load `i18n`** - Don't import `i18n` synchronously at app startup
2. **Exclude `i18n` from `misc` chunk** - Let Vite bundle it with dynamic import
3. **Bundle `React` in main entry chunk** - Guarantee `React` loads before any other chunk

Step-by-Step Fix

1. Remove Synchronous `i18n` Import

File: `src/main.tsx`

Before:

```
import '@/styles/index.css';
import '@/i18n/config'; // ❌ Synchronous import
```

After:

```
import '@/styles/index.css';
// i18n will be initialized lazily in App.tsx to avoid loading before React
```

Why: Prevents i18n from being bundled at entry point, which would force it into a chunk that might load before React.

2. Add Dynamic i18n Import

File: src/App.tsx

Add this to the App component:

```
function App() {
  // Initialize i18n lazily after React is fully loaded
  useEffect(() => {
    // Dynamic import ensures i18n loads AFTER React and all contexts are ready
    import('@/i18n/config').catch((error) => {
      console.error('Failed to initialize i18n:', error);
    });
  }, []);

  // ... rest of component
}
```

Why:

- Dynamic import creates a separate chunk that only loads when `useEffect` runs
 - `useEffect` only runs after React is initialized and mounted
 - Guarantees React is available before i18n code executes
-

3. Update LanguageContext Safety Checks

File: src/contexts/LanguageContext.tsx

Update the `useEffect` in `LanguageProvider`:

```

useEffect(() => {
  // Initialize language from localStorage or browser preference
  const initializeLanguage = () => {
    const savedLanguage = localStorage.getItem('language') as Language;
    if (savedLanguage) {
      setCurrentLanguage(savedLanguage);
      // Check if i18n is initialized before calling changeLanguage
      if (i18n.isInitialized) {
        i18n.changeLanguage(savedLanguage);
      }
      setIsRTL(savedLanguage === 'ar');
      document.documentElement.dir = savedLanguage === 'ar' ? 'rtl' : 'ltr';
      document.documentElement.lang = savedLanguage;
    } else {
      // Default to English
      setCurrentLanguage('en');
      if (i18n.isInitialized) {
        i18n.changeLanguage('en');
      }
      setIsRTL(false);
      document.documentElement.dir = 'ltr';
      document.documentElement.lang = 'en';
    }
  };

  // If i18n is already initialized, use it immediately
  if (i18n.isInitialized) {
    initializeLanguage();
  } else {
    // Otherwise, wait for i18n to be ready
    i18n.on('initialized', initializeLanguage);
    return () => {
      i18n.off('initialized', initializeLanguage);
    };
  }
}, []);

```

Update the changeLanguage function:

```

const changeLanguage = (lang: Language) => {
  setCurrentLanguage(lang);
  // Only call i18n.changeLanguage if i18n is initialized
  if (i18n.isInitialized) {
    i18n.changeLanguage(lang);
  }
  localStorage.setItem('language', lang);
  setIsRTL(lang === 'ar');
  document.documentElement.dir = lang === 'ar' ? 'rtl' : 'ltr';
  document.documentElement.lang = lang;
};

```

Why: Prevents calling i18n methods before it's initialized, avoiding errors during the transition period.

4. Exclude i18n from Misc Chunk

File: vite.config.ts

Add this BEFORE the catch-all rule (around line 227):

```
// i18n - Explicitly exclude from misc chunk to bundle with dynamic import
// Returning undefined allows Vite to bundle these with their importer (App.tsx dynamic import)
// This ensures i18n only loads when useEffect runs, after React is initialized
if (id.includes('node_modules/i18next') ||
    id.includes('node_modules/react-i18next') ||
    id.includes('node_modules/i18next-browser-languagedetector') ||
    id.includes('node_modules/i18next-http-backend')) {
  return undefined; // Bundle with dynamic import chunk, not misc
}

// Everything else from node_modules goes to misc
if (id.includes('node_modules')) {
  return 'misc';
}
```

Why:

- Without this, the catch-all rule (if (id.includes('node_modules')) forces i18n into misc chunk
- Returning undefined tells Vite to bundle i18n with its importer (the dynamic import)
- Creates a separate lazy-loaded chunk that only loads when needed

5. Bundle React in Main Entry Chunk

File: vite.config.ts

Find the React chunking rule (around line 151-161):

Before:

```
// Core React - must load first
if (id.includes('node_modules/react') || id.includes('node_modules/react-dom')) {
  if (!id.includes('react-router') && !id.includes('react-redux') &&
      !id.includes('react-hook-form') && !id.includes('react-query') &&
      !id.includes('react-icons') && !id.includes('react-chartjs') &&
      !id.includes('react-leaflet') && !id.includes('react-i18next') &&
      !id.includes('react-helmet') && !id.includes('react-hot-toast') &&
      !id.includes('react-stripe')) {
    return 'vendor'; // ✗ Separate chunk, loads in parallel
  }
}
```

After:

```
// Core React - Bundle in main entry chunk to guarantee it loads first
// This prevents race conditions where other chunks load before React is available
if (id.includes('node_modules/react') || id.includes('node_modules/react-dom')) {
  if (!id.includes('react-router') && !id.includes('react-redux') &&
      !id.includes('react-hook-form') && !id.includes('react-query') &&
      !id.includes('react-icons') && !id.includes('react-chartjs') &&
      !id.includes('react-leaflet') && !id.includes('react-i18next') &&
      !id.includes('react-helmet') && !id.includes('react-hot-toast') &&
      !id.includes('react-stripe')) {
    return undefined; // ☑ Main entry chunk - always loads first
  }
}
```

Why:

- Main entry chunk is the only chunk guaranteed to load before all others
- By bundling React there, it's **always** available when any other chunk executes
- Eliminates the race condition at the browser level

How It Works Together

Load Sequence (After Fix)

```

1. Browser starts loading page
  ↓
2. Main entry chunk loads (contains React + polyfills)
  ☒ React is now available
  ↓
3. Other chunks load in parallel (misc, router, state, etc.)
  ☒ All can safely use React APIs
  ↓
4. React renders App component
  ↓
5. useEffect runs in App component
  ↓
6. Dynamic import loads i18n chunk
  ☒ React already available, no error
  ↓
7. App fully initialized

```

Chunk Structure (After Fix)

Chunk	Contents	Load Time	React Available?
index-*.js (main)	Polyfills, React, ReactDOM, App code	First (always)	☒ Yes (bundled here)
misc-*.js	Other node_modules (not i18n)	Parallel with others	☒ Yes (main loaded first)
router-*.js	React Router	Parallel	☒ Yes
state-*.js	Redux	Parallel	☒ Yes
[dynamic]-*.js	i18n libraries	Only when useEffect runs	☒ Yes (loaded after mount)

Verification

1. Build the Project

```
npm run build
```

Expected output:

```

☒ Build successful!
☒ Build Summary:
  Total size: ~5.95 MB
  JavaScript: ~2.97 MB (91 files) # Note: One less file (no separate vendor chunk)
  CSS: ~165 KB (2 files)

```

2. Deploy

Push changes and deploy to your hosting platform:

```

git add .
git commit -m "Fix createContext error - bundle React in main chunk"
git push

```

3. Test in Browser

1. **Clear cache:** Ctrl+Shift+R (or Cmd+Shift+R on Mac)
2. **Open Dev Tools Console** (F12)
3. **Reload page**

Expected console output (success):

```

[Inline Polyfill]  Starting synchronous global patching...
[Inline Polyfill] ☒ window.global = window
[Inline Polyfill] ☒ Patched 3 properties across 1 global objects
[Inline Polyfill] ☒ Destructuring test: PASSED
[Inline Polyfill]  Complete - Module scripts can now load safely

```

Should NOT see:

```

✗ Uncaught TypeError: Cannot read properties of undefined (reading 'createContext')

```

4. Verify Chunk Loading

In DevTools Network tab:

1. **Filter by JS files**
2. **Check load order:**
 - `index-*.js` should load first
 - Other chunks load after
 - No `vendor-*.js` (React now in index)

Troubleshooting

Error Still Occurs

If you still see the `createContext` error after applying all fixes:

1. Clear ALL Caches

```
# Local development
rm -rf node_modules/.vite dist
npm install
npm run build

# Browser
- Clear site data in DevTools
- Use incognito/private window
- Hard refresh (Ctrl+Shift+F5)

# CI/CD (Netlify, Vercel, etc.)
- Clear build cache in dashboard
- Trigger new deploy
```

2. Verify Vite Config

Check `vite.config.ts` has all changes:

```
// Should return undefined (not 'vendor')
if (id.includes('node_modules/react') || id.includes('node_modules/react-dom')) {
  // ... exclusions ...
  return undefined; // ☒ Check this is undefined, not 'vendor'
}

// Should return undefined (not 'misc')
if (id.includes('node_modules/i18next') ||
  id.includes('node_modules/react-i18next') ||
  id.includes('node_modules/i18next-browser-languagedetector') ||
  id.includes('node_modules/i18next-http-backend')) {
  return undefined; // ☒ Check this exists and returns undefined
}
```

3. Check Bundle Output

Inspect `dist/` after build:

```
npm run build
ls -lh dist/js/
```

Look for:

- ☒ **Should see:** `index-[hash].js` (larger, contains React)
- ☒ **Should NOT see:** `vendor-[hash].js`

4. Inspect the Error Chunk

If error still occurs in `misc-*.js`:

1. Open the failing `misc-*.js` file in browser DevTools
2. Search for `createContext`
3. Identify which library is calling it
4. Add that library to the exclusion list in `vite.config.ts`

Example:

```
if (id.includes('node_modules/problematic-library')) {  
  return undefined; // Exclude from misc chunk  
}
```

Why Previous Attempts Failed

Attempt 1: Dynamic i18n Import Only

What we tried: Load i18n lazily in `useEffect`

Why it failed:

- Catch-all rule still forced i18n into misc chunk
- misc chunk loaded before React chunk
- Still hit the race condition

Attempt 2: Remove i18n from Manual Chunking

What we tried: Removed explicit `return 'misc'` for i18n

Why it failed:

- Catch-all rule (`if (id.includes('node_modules')) return 'misc'`) still caught i18n
- Same race condition persisted

Attempt 3: Exclude i18n from Misc Chunk

What we tried: Return `undefined` for i18n before catch-all

Why it failed:

- i18n no longer in misc chunk ☒
- But misc chunk still loaded in parallel with vendor chunk
- Race condition between vendor (React) and misc (other libs using React)

Final Solution: React in Main Entry Chunk

What works: Bundle React in main entry chunk

Why it works:

- Main chunk **always** loads first (browser guarantee)
- No parallel loading of React
- No race condition possible
- All other chunks execute after React is available

Technical Deep Dive

Vite Chunk Loading Mechanics

Vite generates code like this:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <!-- Main entry chunk - loads first -->  
    <script type="module" src="/js/index-abc123.js"></script>  
  </head>  
  <body>  
    <div id="root"></div>  
  </body>  
</html>
```

Inside `index-abc123.js`:

```
// Dynamic imports create parallel requests
import('./misc-xyz789.js'); // Loads in parallel
import('./router-def456.js'); // Loads in parallel
// ... etc
```

Problem: All dynamic imports start loading **immediately and in parallel**. No guaranteed order.

Solution: Bundle critical dependencies (React) in the main chunk, not in dynamic imports.

Module Parse vs. Execution

```
// This runs at PARSE time (when file loads):
import { createContext } from 'react';
const MyContext = createContext(null); // ✗ Runs immediately

// This runs at EXECUTION time (when called):
export function useMyContext() {
  return useContext(MyContext); // ☑ Runs later
}
```

When `misc` chunk is parsed before React chunk loads, the `parse-time` `createContext` call fails.

Why undefined (Main Chunk) Works

In `manualChunks`:

- `return 'vendor'` → Creates separate `vendor-*.js` chunk
- `return 'misc'` → Creates separate `misc-*.js` chunk
- `return undefined` → Bundles in main `index-*.js` chunk

Main chunk loads via `<script>` tag in HTML, which the browser processes **before** any dynamic imports.

Prevention for Future

Best Practices

1. **Bundle React in main chunk** - Never split React into a separate chunk
2. **Lazy load heavy dependencies** - Use dynamic imports for large libraries
3. **Check `i18n.isInitialized`** - Always verify before calling `i18n` methods
4. **Test chunk load order** - Use Network tab to verify load sequence
5. **Monitor bundle size** - Main chunk should include critical dependencies

Recommended Vite Config Pattern

```
manualChunks: (id) => {
  // Critical dependencies → main chunk (undefined)
  if (id.includes('node_modules/react') ||
      id.includes('node_modules/react-dom')) {
    return undefined;
  }

  // Lazy-loaded dependencies → exclude from catch-all
  if (id.includes('node_modules/i18next') ||
      id.includes('node_modules/react-i18next')) {
    return undefined;
  }

  // Feature-specific chunks
  if (id.includes('node_modules/react-router')) {
    return 'router';
  }

  // Catch-all for other node_modules
  if (id.includes('node_modules')) {
    return 'misc';
  }
}
```

Summary

The Error

Cannot read properties of undefined (reading 'createContext') in misc chunk

Root Cause

Race condition: misc chunk loading/parsing before React chunk loads

Solution

Three-part fix:

1. Lazy load i18n via dynamic import in `useEffect`
2. Exclude i18n from misc chunk (`return undefined`)
3. Bundle React in main entry chunk (`return undefined`)

Why It Works

Main chunk loads first → React available → All other chunks can safely use React APIs

Files Changed

- `src/main.tsx` - Remove synchronous i18n import
- `src/App.tsx` - Add dynamic i18n import
- `src/contexts/LanguageContext.tsx` - Add `i18n.isInitialized` checks
- `vite.config.ts` - Exclude i18n, bundle React in main chunk

Additional Resources

- [Vite Code Splitting Docs \(https://vitejs.dev/guide/build.html#chunking-strategy\)](https://vitejs.dev/guide/build.html#chunking-strategy)
- [React Context API \(https://react.dev/reference/react/createContext\)](https://react.dev/reference/react/createContext)
- [i18next Initialization \(https://www.i18next.com/overview/api#init\)](https://www.i18next.com/overview/api#init)
- [Module Loading Order in Browsers \(https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules\)](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules)

Document Version: 1.0

Last Updated: 2025-10-01

Status: ☒ Verified Working